Constant Propagation

1 Introduzione

> L'obiettivo della **constant propagation** è quello di determinare in quali punti del programma le variabili hanno un valore costante. In questo modo è possibile sostituire **tutti gli usi di queste variabili** con i rispettivi valori costanti, a patto che non vengano riassegnate con espressioni non costanti.

1.1 Esempio di applicazione della pura Constant Propagation

```
b = 3
c = 1 + b
d = b + c

=> Sostituisco gli usi di b con il suo valore, cioè 3

b = 3
c = 1 + 3
d = 3 + c
```

In questo snippet di codice le variabili `c` e `d` hanno come operando la variabile `b`.
Poiché a `b` è assegnato un valore costante, e quindi già noto a tempo di compilazione, il compilatore può sostituire tutti gli usi di `b` in `c` e `d`.

1.2 Constant Propagation + Constant Folding

Osservando il codice sorge una domanda. ***L'applicazione della sola constant propagation contribuisce a ottimizzare il codice?***

La risposta a questo quesito è negativa poiché la sola constant propagation non è altro che un cambio di simbolo da parte del compilatore.
Difatti, esaminando per esempio la seconda istruzione, il suo risultato verrebbe comunque calcolato dalla CPU a runtime, pertanto **la sola constant propagation non ha ottimizzato l'attuale IR**.

Per ottenere una concreta ottimizzazione è prassi combinare la constant propagation con altre tipologie di passi di ottimizzazione, come la **constant folding**.

> Tramite la **constant folding** si può **valutare un espressione costante a tempo di compilazione**, ovvero determinare fin da subito il risultato di un espressione aritmetica / booleana, senza attendere la fase di runtime.

Attraverso l'applicazione iterativa di questa coppia di ottimizzazioni si riesce a raggiungere due importanti obiettivi:

- **Propagare il risultato di un espressione** ad eventuali istruzioni successive che lo usano.
- Diminuire il numero di calcoli effettuati dalla CPU in runtime, **abbassando di conseguenza il tempo d'esecuzione richiesto per l'esecuzione del programma**.

1.2.1 Esempio

```
b = 3
c = 1 + b
d = b + c

=> Sostituisco gli usi di b con il suo valore, cioè 3

b = 3
c = 1 + 3    // Applico la constant folding, 1+3 = 4
d = 3 + c

=>

b = 3
c = 4
d = 3 + c

=> Sostituisco gli usi di c con il suo nuovo valore, cioè 4

b = 3
c = 4
d = 3 + 4  // Riapplico la constant folding, 3 + 4 = 7

=>

b = 3
c = 4
d = 7
```

Attraverso l'applicazione iterativa di questo processo si è riusciti a tempo di compilazione di evitare il calcolo di ben due operazioni aritmetiche alla fase di runtime.

Eventualmente, poi, in base alla caratteristiche del programma, tali variabili potrebbero essere eliminate attraverso l'uso della **dead code elimination** in caso di inutilizzo.

---

2 Framework

Il primo requisito per la compilazione del framework è stabilire qual è il **dominio di riferimento**.

Per cominciare, presentiamo una veloce rappresentazione dei passaggi mentali che si compiono quando si tenta di applicare tale ottimizzazione a mente:

- `k = 2` **- Assegnamento di un valore costante**
  - Uccide la precedente definizione di `k`
  - Possibile applicazione della constant propagation in futuro
- `a = k + 2` **- Constant Propagation**
  - Uccide la precedente definizione di `a`
  - Verificare l'esistenza della variabile `k` e del suo eventuale valore. Se costante, applicare la constant propagation
- `a = 2 + 2` **- Constant Folding**
  - Uccide la precedente definizione di `a`
  - Si valuta l'espressione e assegno il risultato alla variabile `a`

È quindi evidente quale sia il contenuto informativo, ovvero il **dominio**, da considerare. L'informazione da calcolare per ogni **nodo n del CFG** è un **insieme di coppie del tipo** `variabile, valore costante>`.

Per determinare quale sia la **direzione di analisi**, bisogna **comprendere quando una certa coppia è disponibile in un nodo** `p`.

> Una coppia $\langle x, k \rangle$ definita nel nodo `d` è applicabile nel nodo `p` (in cui è presente un suo uso) se tra tutti i possibili percorsi per raggiungere `p` da `d` la coppia `<x,k>` non viene ridefinita.

Quindi, per determinare se una coppia $\langle x, k \rangle$ è valida in un nodo `p`, si deve verificare che essa sia stata definita in un'istruzione precedente e che non sia stata sovrascritta lungo alcun percorso che porta a `p`.
Poiché la validità di un'informazione dipende esclusivamente dalle istruzioni eseguite nel **passato** rispetto al nodo corrente, la direzione dell'analisi è di tipo **Forward**.

Sia `b` una generica istruzione del CFG, come sono definiti i suoi `in[b] e out[b]`?

`in[b]` è l'insieme delle coppie che sono disponibili all'inizio dell'istruzione `b` indipendentemente dal percorso preso per raggiungerla. Per garantire la correttezza dell'ottimizzazione, si utilizza l'operatore di **meet** $(\wedge)$, che in questo contesto corrisponde all'**intersezione degli insiemi di coppie**.

La scelta è motivata dall'esigenza che si deve avere l'assoluta certezza che una certa definizione di variabile raggiunga un certo nodo nel caso in cui dalla definizione si propagano più percorsi che convergono in quel punto. Mediante l'uso dell'operatore di intersezione si riesce a propagare soltanto le definizioni presenti in tutti i percorsi.

L'uso dell'intersezione assicura che una variabile sia considerata costante in $in[b]$ solo se tutti i predecessori concordano sullo stesso valore costante. Se un ramo fornisse $\langle x, 5 \rangle$ e un altro $\langle x, 6 \rangle$ (o non fornisse alcuna costante per x), la coppia verrebbe esclusa dall'intersezione, indicando che x non è una costante affidabile in quel punto.

In conclusione, definiamo quindi `in[b]` come:

$$in[b] = \wedge\, out[pred(b)]$$

dove $\wedge$ corrisponde all'operatore di intersezione $\cap$.

`out[b]` invece rappresenta l'insieme delle coppie di variabili disponibili per la prossima istruzione. Il risultato memorizzato in `out[b]` è l'output della funzione di trasferimento applicata all'input di `b`.

$$out[b] = f_b(in[b])$$

Ragioniamo ora su come arrivare a definire l'equazione per la funzione di trasferimento.

Prima di tutto si procede a definire l'insieme di partenza e l'insieme degli `out[b]` per ogni istruzione `b` eccetto l'entry point.

- **Boundary Condition:** $out[entry] = \emptyset$, poiché all'inizio del programma ancora nessuna variabile locale è stata inizializzata.
- **Initial Interior Points:** Per tutte le altre istruzioni `b` si imposta all'inizio $out[b] = \mathbb{U}$ (dove $\mathbb{U}$ è l'insieme di tutte le possibili coppie <variabili, costanti>).
  - Perché l'intersezione ($\cap$) è un operatore "distruttivo". Se si inizializzassero con $\emptyset$, l'intersezione tra $\emptyset$ e qualsiasi altra cosa darebbe sempre $\emptyset$, e non si otterrebbero mai delle costanti.

La funzione di trasferimento f_b ha il compito di:

- **Generare** nuove coppie se `b` è un'assegnazione a una costante
- **Rimuovere** (kill) coppie preesistenti se la variabile viene ridefinita con un valore non costante o ignoto
- **Valutare** espressioni se gli operandi sono già costanti in `in[b]`, ovvero applicare la *constant folding*
  - `z = x + 2` e `x = 5` $\rightarrow \langle z, 7 \rangle$

Dunque, la funzione di trasferimento f_b è così definita:

$$f_b(in[b]) = Gen_b(in[b]) \ \cup (in[b] \setminus Kill_b)$$

$(in[b] \setminus Kill_b)$
A seguito della nuova definizione di `b`, viene eliminata dall'insieme di input `in[b]` la sua precedente versione.

Inoltre, la definizione di Gen_b è diversa rispetto a quella vista finora:

$$Gen_b(in[b]) = \{\langle x, val(e, in[b]) \rangle \mid val(e, in[b]) = valore\ costante\}$$

Dove la funzione $val(e, in[b])$ è definita come:

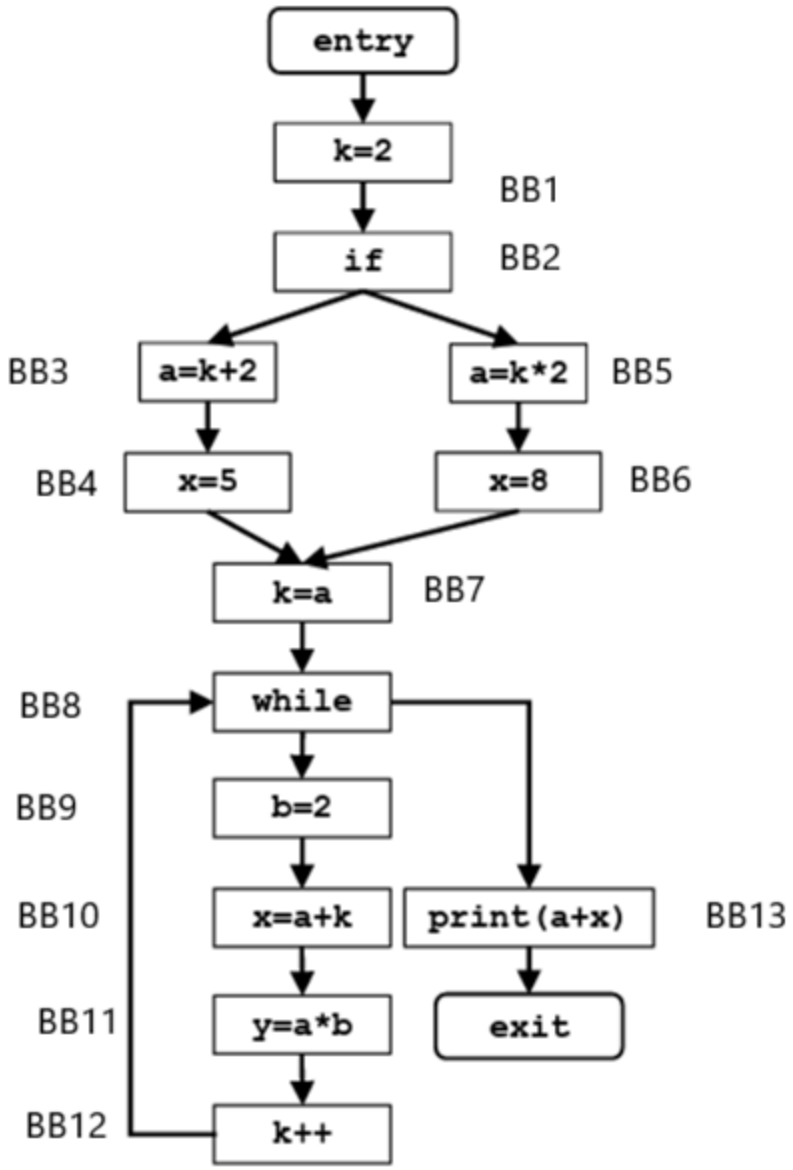
- Se e è una costante k, allora $val = k$.
- Se e è una variabile y e $\langle y, k \rangle \in in[b]$, allora $val = k$.
- Se e è un'operazione $y_1 \text{ op } y_2$, e sia y_1 che y_2 hanno valori costanti in $in[b]$, allora $val = k_1 \text{ op } k_2$.
- Altrimenti val è uguale ad un valore non costante

	Constant Propagation
Domain	Insieme di coppie del tipo <variabile, valore costante>

	Constant Propagation
Direction	Forward $out[b] = f_b(in[b])$ $in[b] = \wedge\ out[pred(b)]$
Transfer function	$f_b(in[b]) = Gen_b(in[b])\ \cup (in[b] \setminus Kill_b)$
Meet Operation ($\wedge$)	$\cap$
Boundary Condition	$out[entry] = \emptyset$
Initial interior points	$out[exit] = \mathbb{U}$

---

3. Tabella delle iterazioni


entry
k=2
BB1
if
BB2
BB3
a=k+2
a=k*2
BB5
BB4
x=5
x=8
BB6
k=a
BB7
BB8
while
BB9
b=2
BB10
x=a+k
print(a+x)
BB13
BB11
y=a*b
exit
BB12
k++

	Iterazione 1		Iterazione 2		Iterazione 3	
	IN[B]	OUT[B]	IN[B]	OUT[B]	IN[B]	OUT[B]
BB1	Vuoto	{(k,2)}	Vuoto	{(k,2)}	Vuoto	{(k,2)}
BB2	{(k,2)}	{(k,2)}	{(k,2)}	{(k,2)}	{(k,2)}	{(k,2)}
BB3	{(k,2)}	{(k,2), (a,4)}	{(k,2)}	{(k,2), (a,4)}	{(k,2)}	{(k,2), (a,4)}
BB4	{(k,2), (a,4)}	{(k,2), (a,4), (x,5)}	{(k,2), (a,4)}	{(k,2), (a,4), (x,5)}	{(k,2), (a,4)}	{(k,2), (a,4), (x,5)}
BB5	{(k,2)}	{(k,2), (a,4)}	{(k,2)}	{(k,2), (a,4)}	{(k,2)}	{(k,2), (a,4)}
BB6	{(k,2), (a,4)}	{(k,2), (a,4), (x,8)}	{(k,2), (a,4)}	{(k,2), (a,4), (x,8)}	{(k,2), (a,4)}	{(k,2), (a,4), (x,8)}
BB7	{(k,2), (a,4)}	{(k,4), (a,4)}	{(k,2), (a,4)}	{(k,4), (a,4)}	{(k,2), (a,4)}	{(k,4), (a,4)}
BB8	{(k,4), (a,4)}	{(k,4), (a,4)}	{(a,4)}	{(a,4)}	{(a,4)}	{(a,4)}
BB9	{(k,4), (a,4)}	{(k,4), (a,4), (b,2)}	{(a,4)}	{(a,4), (b,2)}	{(a,4)}	{(a,4), (b,2)}
BB10	{(k,4), (a,4), (b,2)}	{(k,4), (a,4), (b,2), (x,8)}	{(a,4), (b,2)}	{(a,4), (b,2)}	{(a,4), (b,2)}	{(a,4), (b,2)}
BB11	{(k,4), (a,4), (b,2), (x,8)}	{(k,4), (a,4), (b,2), (x,8), (y,8)}	{(a,4), (b,2)}	{(a,4), (b,2), (y, 8)}	{(a,4), (b,2)}	{(a,4), (b,2), (y, 8)}
BB12	{(k,4), (a,4), (b,2), (x,8), (y,8)}	{(k,5), (a,4), (b,2), (x,8), (y,8)}	{(a,4), (b,2), (y, 8)}	{(a,4), (b,2), (y, 8)}	{(a,4), (b,2), (y, 8)}	{(a,4), (b,2), (y, 8)}
BB13	{(k,4), (a,4)}	{(k,4), (a,4)}	{(a,4)}	{(a,4)}	{(a,4)}	{(a,4)}